数据结构与算法 A

2022 年期中 · 试卷

考试日期：2022 年 10 月 26 日。按原始试题 PDF 的题目部分整理，省略考场纪律与装订封面。原卷第一部分标“8 小题”，实际可见主编号为 1—7，分值合计 20 分；本卷保留实际编号。

姓名：　　学号：　　任课教师：

一、选择填空（20 分）

1．（2 分）数据结构核心三要素不包含（　）。

A. 逻辑结构　B. 存储结构　C. 时间复杂度　D. 运算

2．（2 分）以下算法的时间复杂度是（　）。提示：调和级数 $\sum_{i=1}^{n}\frac{1}{i}=\ln n+O(1)$。

```
int m = 1, i, j;
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j += i)
        m *= 2;
```

A. $\Theta(\log_2 n)$　B. $\Theta(n\log_2 n)$　C. $\Theta(n)$　D. $\Theta(n^2)$

3．（2 分）关于顺序表和链表的应用场合，下列说法不正确的是（　）。

A. 经常插入删除时，不宜使用顺序表

B. 无法确定线性表长度最大值时，不宜使用链表

C. 当读操作比插入删除操作频率大时，不应选择链表

D. 当指针开销占结点空间比例较大时，应慎重选择链表

4．（2 分）双循环链表中，在结点 p 后插入新结点 s，以下做法不正确的是（　）。

```
// A
p->next->prev=s; s->next=p->next;
p->next=s; s->prev=p;
// B
p->next->prev=s; p->next=s;
s->next=p->next; s->prev=p;
// C
s->next=p->next; p->next->prev=s;
s->prev=p; p->next=s;
```

5．（3 分）abcde 顺序入栈，哪些不是可行的出栈序列？（可多选；都可行则写“无”）

A. cbdae　B. cabed　C. decab　D. dceab

（3 分）所有可行的出栈序列共有　　种。

6．（2 分）值为 1 至 4 的四个结点能组成　　种不同的二叉搜索树。

（2 分）将 {12,70,33,65,24,56,48,92,86,33} 按 $O(n)$ 建堆方法调整为最小值堆，写出序列。

7．（2 分）树 T 的度为 4，度为 1、2、3、4 的结点数分别为 4、2、1、1。叶子结点有（　）个。

A. 4　B. 6　C. 8　D. 10

二、辨析与简答（30 分）

1．（3 分）顺序栈和链式栈底层分别以定长数组和链表存储数据。对比二者的时间效率与空间效率。

2．对于字符串 `1212123`：

（1）（3 分）按基于 ASCII 编码的字典序，从小到大写出全部不同的非空子串。

（2）（4 分）写出优化后的 next 特征向量。

3．（6 分）二叉树的前、中、后序序列如下，部分字符未标出。构造并画出该二叉树，补全三个序列。每个下划线表示一个结点。

```
前序：Z _ C V U F _ Y _ W
中序：C _ U V Z _ G F I _
后序：C _ _ B Y G W I _ _
```

4．消息中字母 a、b、c、d、e、f 的频数依次为 5、9、12、13、16、45。

（1）（3 分）采用 Huffman 编码，画出树。

（2）（2 分）平均编码长度是多少？

（3）（2 分）相对 6 个字母的等长编码，节省多少空间？

5．并查集以父指针数组表示，非根存父下标，根存所在集合大小的相反数。元素 0—8 的数组为 `[1,-4,3,1,-3,4,4,8,-2]`。

（1）（4 分）画出森林。

（2）（3 分）采用重量权衡合并与路径压缩，合并元素 2 和 7 所在集合，写出合并后的数组。

三、算法填空（每空 2 分，共 10 分）

给定树结点定义，getChildrenNum 和 getKthChild 已实现；root 无兄弟结点。实现非递归后根遍历。子结点按从左到右编号 0 至 n-1。

原卷构造函数写作 rd=rd，不能正确给成员赋值；此处修正为 this->rd=rd。同步统一 Node/TreeNode 与函数大小写。

```
class TreeNode {
public:
    int getChildrenNum();
    TreeNode* getKthChild(int k);
};
class stackElement {
public:
    TreeNode* node;
    int rd;
    stackElement(TreeNode* nd, int rd) {
        node = nd;
        this->rd = rd;
    }
};
void postOrderTraversal(TreeNode* root) {
    stack<stackElement> st;
    st.push(stackElement(root, 0));
    while (/* (1) */) {
        stackElement topElem = st.top();
        /* (2) */;
        if (topElem.node->getChildrenNum() == 0 || /* (3) */) {
            visit(topElem.node);
        } else {
            st.push(stackElement(topElem.node, 1));
            for (int i=0; i<topElem.node->getChildrenNum(); ++i) {
                TreeNode* child = topElem.node->getKthChild(/* (4) */);
                st.push(/* (5) */);
            }
        }
    }
}
```

四、算法设计与实现（30 分）

1．（10 分）两个先进先出队列 A、B，允许的操作只有：front() 返回队首、push_back(x) 插入队尾、pop_front() 删除队首、empty() 判断空。用 A、B 模拟栈的 top、pop、push、empty 四个操作。
简述思想并给出伪代码，无时间限制，额外空间不超过 $O(1)$；题目假定递归不占空间。top/pop 时栈非空，不考虑溢出。

2．（10 分）判断字符串 S 是否含形如 AA 的子串，其中 A 是非空字符串。若有，返回这种子串第一次出现的起始下标（从 0 开始）。例如 `dabcabc` 返回 1。
给出算法流程，说明正确性并分析复杂度。尽量达到 $O(n^2)$ 或更优。

3．（10 分）给定二叉树 root 与整数 targetSum，求结点值之和等于 targetSum 的路径数，并分析时间复杂度。路径可不从根开始、可不在叶子结束，但只能由父到子向下。
例树（左右孩子按括号顺序，空孩子写 ∅）：

```
10:(5,-3)  5:(3,2)  -3:(∅,11)
3:(3,-2)   2:(∅,1)
```

当 targetSum=8 时共有 3 条符合要求的路径。

五、分析证明（10 分）

1．非空 k 叉树（$k \geq 2$）中，每个非叶子结点恰有 k 个孩子。
（1）（4 分）有 m 个非叶子结点，求叶子结点数并推导。
（2）（6 分）高度为 h（独根树高 1），结点最多和最少各多少？给出推导。